

Enterprise Cloud-Native DevOps Platform

Complete End-to-End Project Documentation

React

Java

Python

Docker

Kubernetes

Terraform

Azure DevOps

GitHub

PREPARED FOR MANAGEMENT REVIEW

Microservices | CI/CD Automation | Container Orchestration | IaC

May 2026

CONFIDENTIAL - INTERNAL USE ONLY



Table of Contents

1.	Enterprise Architecture Overview	3
2.	Technology Stack	4
3.	Phase 1 – Application Development	5
4.	Phase 2 – Docker Containerisation	6
5.	Phase 3 – Kubernetes Orchestration	7
6.	Phase 4 – Terraform Infrastructure	9
7.	Phase 5 – Azure DevOps CI/CD	10
8.	CI/CD Pipeline Diagram	11
9.	Kubernetes Architecture Diagram	12
10		
.	End-to-End Deployment Lifecycle	13
11		
.	Monitoring & Future Enhancements	14
12		
.	Professional Skills Demonstrated	15
13		
.	Conclusion	16

This project delivers a production-grade enterprise e-commerce platform built on a microservices architecture. Every component has been engineered for scalability, portability, and automated delivery — reflecting the standards of modern technology organisations. The platform spans application development, containerisation, orchestration, infrastructure automation, and continuous delivery via Azure DevOps.

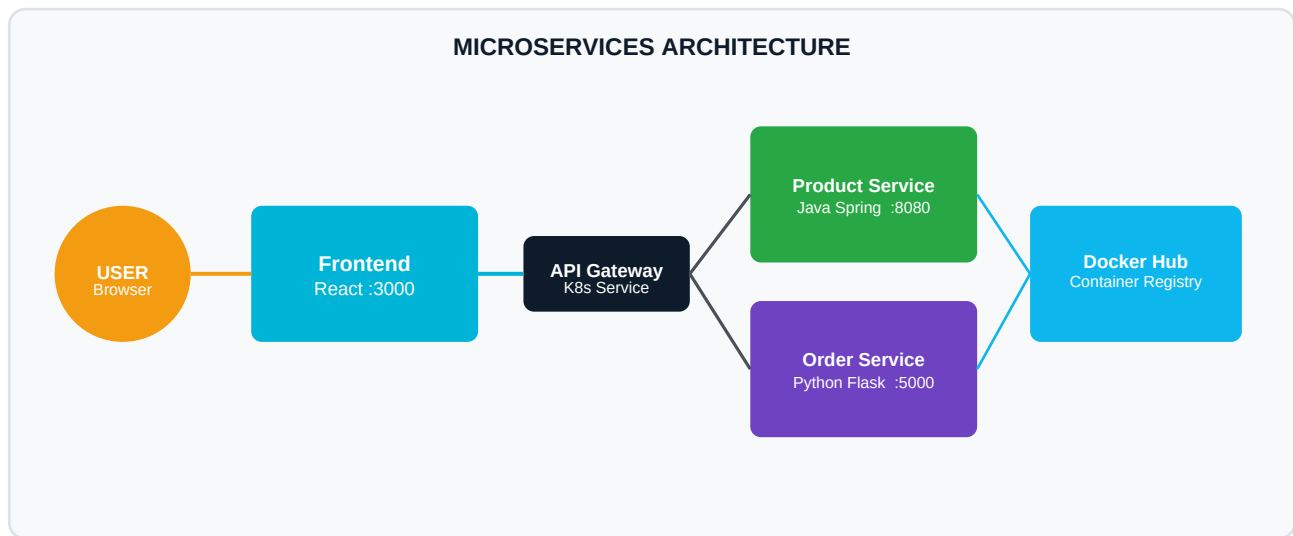


Figure 1 – High-level microservices communication flow

Architecture Components

- GitHub stores all source-code repositories and acts as the single source of truth
- Azure DevOps handles both CI (build, test, image push) and CD (deploy to Kubernetes)
- Docker Hub is the container registry where all application images are published
- Terraform automates all Kubernetes infrastructure provisioning as code
- Kubernetes orchestrates containers, manages replicas, and performs rolling updates
- Frontend (React) and backend services communicate over Kubernetes internal DNS

Component	Technology	Purpose
Frontend	React	User-facing e-commerce UI
Backend – Products	Java Spring Boot	Product catalogue REST API
Backend – Orders	Python Flask	Order processing REST API
Containerisation	Docker	Package services into images
Container Registry	Docker Hub	Store & version Docker images
CI/CD Platform	Azure DevOps	Automated build, test & deploy
Orchestration	Kubernetes	Container scheduling & scaling
Infrastructure as Code	Terraform	Provision K8s resources declaratively
Version Control	GitHub	Source code management

The first phase established three independent services, each with its own technology stack, and validated their local operation before containerisation.

React Frontend Displays the product catalogue and shopping interface. Connects to backend REST APIs over HTTP, rendering data returned as JSON.	Java Product Service Spring Boot REST API exposing GET /products and POST /products . Manages product inventory and retrieval with a clean separation of concerns.	Python Order Service Flask REST API exposing GET /orders and POST /orders . Handles customer order processing independently from the product service.
---	--	---

Key concepts applied:

- REST API design & JSON-based inter-service communication
- Separation of concerns: each service owns a single domain
- Local integration testing before containerisation
- Frontend/backend decoupling for independent deployability

Docker wraps each microservice — together with its runtime dependencies — into a self-contained, portable image. This eliminates the 'it works on my machine' problem and establishes a reproducible artefact that flows unchanged through every environment.

Dockerfile Responsibilities	Docker Compose Responsibilities
Define base runtime image (e.g. openjdk:17-slim)	Coordinate multi-container startup order
Copy compiled application JAR / Python files	Configure internal bridge network
Expose service port	Map host ports to container ports
Set entrypoint / startup command	Declare service dependencies (depends_on)

Container-to-container communication uses Docker DNS: instead of **localhost**, the frontend reaches the product service via **product-service:8080** — the service name acts as the hostname on the shared bridge network.

Benefits Achieved

- Reproducible builds across dev, staging, and production
- Faster developer onboarding — single docker compose up command
- Image versioning and rollback via Docker Hub tags
- Environment standardisation eliminates configuration drift

Kubernetes transforms standalone containers into a production-grade distributed system. It continuously monitors pod health, automatically restarts failed containers, balances traffic across replicas, and executes zero-downtime rolling updates.

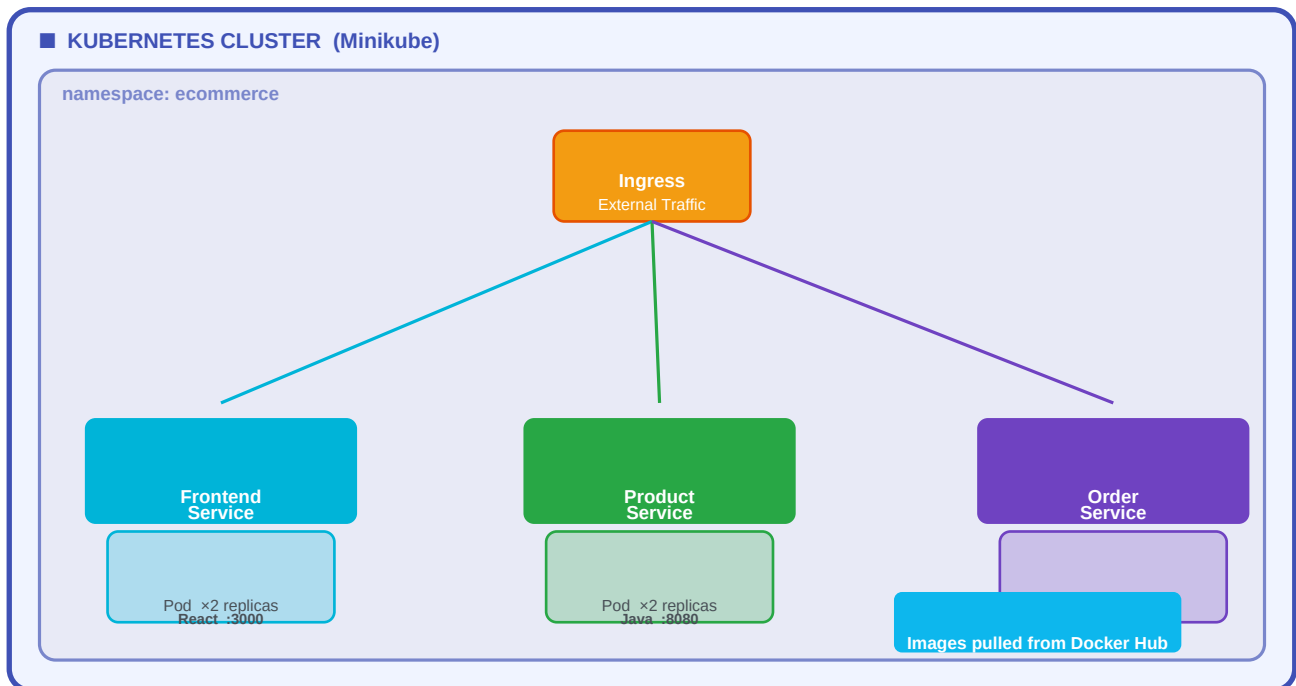


Figure 2 – Kubernetes cluster internal architecture

Resource	Role in This Project
Pod	Smallest deployable unit; runs one container instance per microservice
Deployment	Maintains desired replica count; manages rolling updates & restarts
Service	Provides a stable virtual IP and DNS name for each microservice
NodePort	Exposes the React frontend on a host port for external browser access
Namespace	Logical isolation boundary for all ecommerce workloads

Essential kubectl commands used:

```
kubectl get pods – list all running pods
```

```
kubectl logs – inspect container logs
```

```
kubectl describe pod – detailed pod events & status
```

```
kubectl rollout restart – trigger rolling restart
```

Terraform introduces Infrastructure as Code (IaC) practices: all Kubernetes resources — namespaces, deployments, services — are declared in version-controlled HCL files. This removes manual `kubectl` commands from the deployment process, ensures infrastructure is always reproducible, and enables peer-reviewed infrastructure changes.

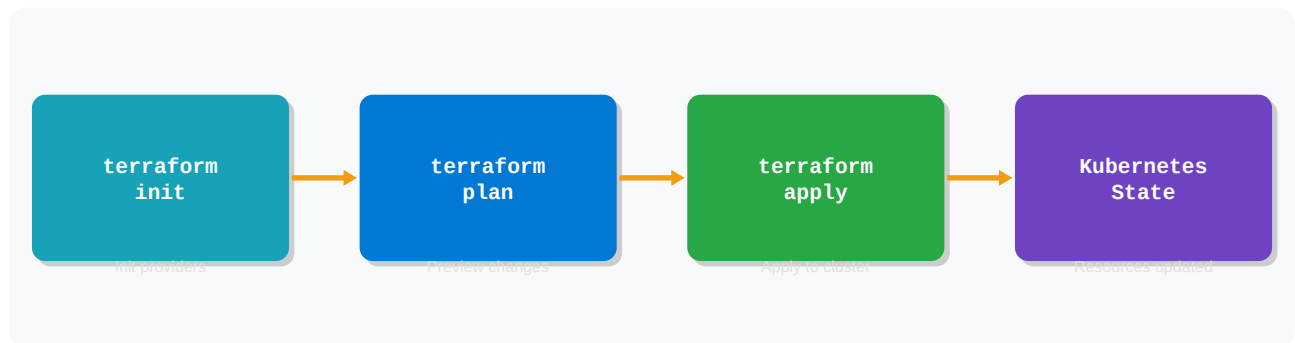


Figure 3 – Terraform workflow: `init` → `plan` → `apply`

Command	Action
<code>terraform init</code>	Download provider plugins (Kubernetes provider)
<code>terraform plan</code>	Preview what resources will be created, changed, or destroyed
<code>terraform apply</code>	Execute the plan and converge real state to desired state
<code>terraform destroy</code>	Tear down all managed resources cleanly

IaC Benefits

- Infrastructure is version-controlled alongside application code in GitHub
- Eliminates configuration drift between environments
- Infrastructure changes go through pull-request review before apply
- Terraform state file tracks what is deployed, preventing duplicate resources

Azure DevOps serves as the single automation platform for both Continuous Integration and Continuous Deployment. A YAML pipeline definition stored in the GitHub repository drives every stage — from source checkout through build and test to container image publication and Kubernetes rollout.

Stage	CI Pipeline (Build & Test)	CD Pipeline (Deploy)
1	Trigger: push/PR to main branch	Trigger: CI pipeline succeeds
2	Checkout source from GitHub	Pull latest images from Docker Hub
3	Build Java (Maven) & Python apps	Run terraform plan against cluster
4	Execute unit & integration tests	Apply Kubernetes deployment manifests
5	Build Docker images for all services	Perform rolling update of pods
6	Push versioned images to Docker Hub	Health-check new pods before cutover

Azure DevOps Advantages

- Single YAML pipeline file — pipeline-as-code reviewed alongside application changes
- Automatic rollback if health checks fail after deployment
- Environment-specific variable groups (dev / staging / prod) in Azure DevOps Library
- Build artefacts and deployment history retained for audit and compliance

The diagram below illustrates the complete automated flow from a developer git push through to running pods in the Kubernetes cluster. Every step is orchestrated by Azure DevOps with no manual intervention required.

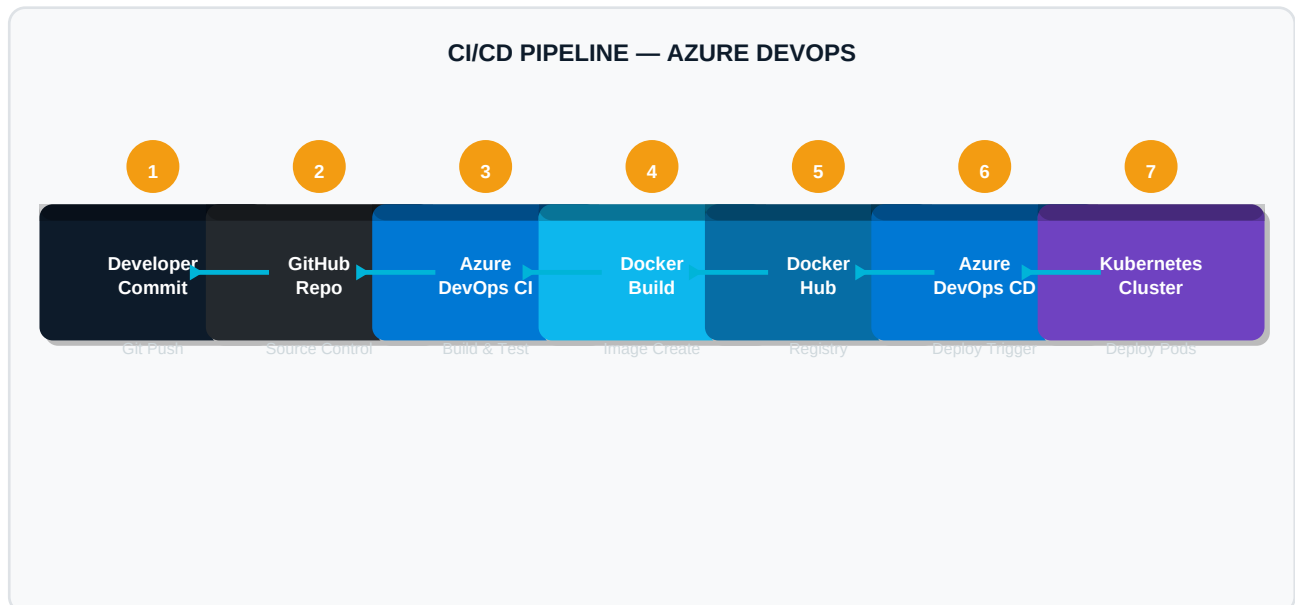


Figure 4 – Full CI/CD pipeline: Developer → GitHub → Azure DevOps → Docker Hub → Kubernetes

Detailed Pipeline Steps:

- 1 Developer pushes code to a feature branch and opens a Pull Request on GitHub
- 2 Azure DevOps CI pipeline triggers automatically on PR creation / push to main
- 3 Pipeline checks out source, compiles Java via Maven, installs Python dependencies
- 4 Unit tests and integration tests execute; pipeline fails fast on test failure
- 5 Docker images are built for all three services (frontend, product, order)
- 6 Images are tagged with git commit SHA and pushed to Docker Hub
- 7 Azure DevOps CD pipeline triggers upon successful CI; pulls latest images
- 8 Terraform validates cluster infrastructure state; applies any drift corrections
- 9 Kubernetes rolling update begins: new pods created, old pods terminated gracefully
- 10 Frontend becomes accessible via NodePort; smoke tests verify the deployment

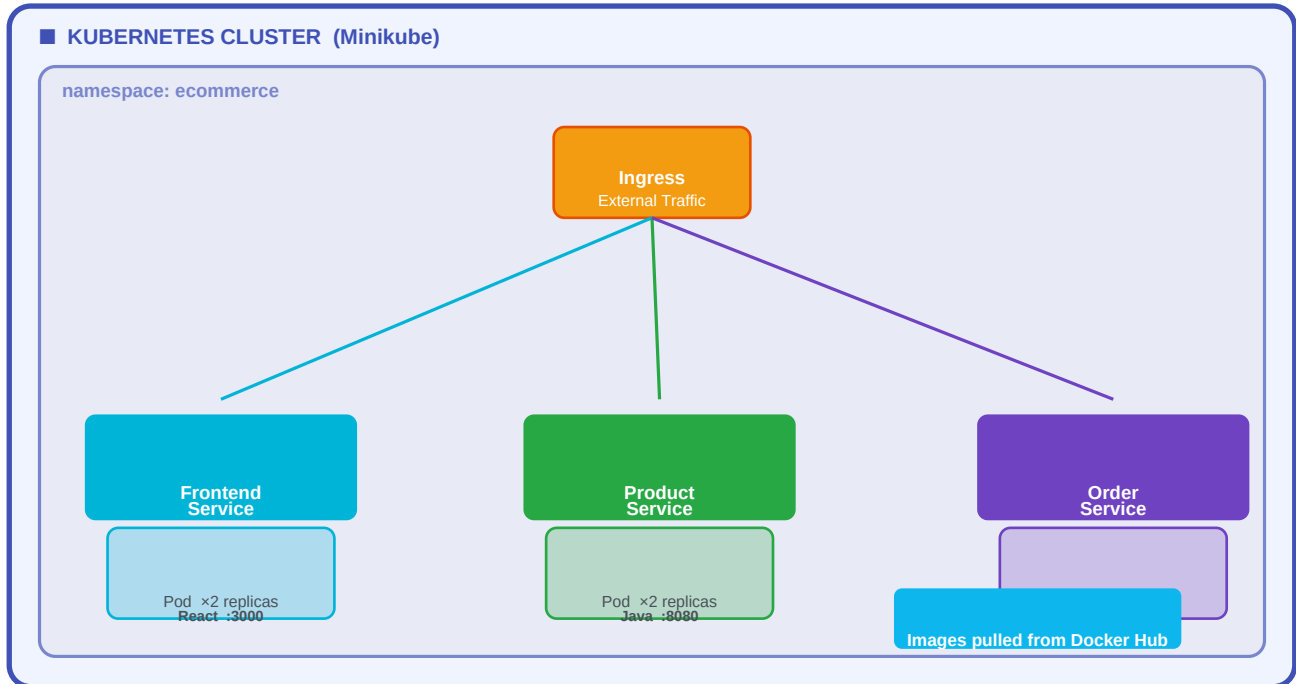


Figure 5 – Kubernetes internal architecture and service routing

The cluster hosts a dedicated **ecommerce** namespace containing three deployments and their corresponding services. An Ingress resource routes external HTTPS traffic to the frontend service, while the frontend communicates with backend services over Kubernetes internal DNS (e.g. `product-service.ecommerce.svc.cluster.local`). Each deployment maintains two pod replicas to ensure high availability.

Communication Path	Protocol	Kubernetes Mechanism
External user → Frontend	HTTPS	NodePort / Ingress
Frontend → Product Service	HTTP/REST	ClusterIP Service DNS
Frontend → Order Service	HTTP/REST	ClusterIP Service DNS
Deployments ← Docker Hub	HTTPS	ImagePullPolicy: Always

Developer	Writes code, runs local unit tests, pushes to feature branch
GitHub	PR triggers code review; merge to main initiates pipeline
Azure DevOps CI	Builds all services, executes tests, creates versioned Docker images
Docker Hub	Receives and stores versioned images; acts as the deployment artefact store
Azure DevOps CD	Detects new images; triggers deployment pipeline automatically
Terraform	Validates and reconciles infrastructure state before deployment
Kubernetes	Executes rolling update; new pods replace old pods with zero downtime
Monitoring	Liveness & readiness probes confirm healthy pod state post-deploy

Observability	Security	Scalability	Advanced DevOps
• Prometheus metrics scraping for all services • Grafana dashboards for CPU, memory, request rate • Alertmanager for on-call notification routing • Distributed tracing with Jaeger / OpenTelemetry	• Kubernetes Secrets for credential management • Image vulnerability scanning (Trivy / Snyk) • Network policies for pod-to-pod traffic control • RBAC role-based access control for the cluster	• Horizontal Pod Autoscaler (HPA) on CPU/memory • Multi-replica deployments for HA • Cluster Autoscaler for node-level scaling • Load testing with k6 / Locust	• Helm charts for parameterised deployments • ArgoCD GitOps continuous reconciliation • SonarQube static code analysis in pipeline • Canary & blue-green deployment strategies

Skill Area	Detail
Microservices Architecture	Designed independently deployable services with clear domain boundaries
REST API Development	Built and tested APIs in Java (Spring Boot) and Python (Flask)
Docker Containerisation	Wrote Dockerfiles, built images, orchestrated with Compose
Kubernetes Orchestration	Deployed pods, services, deployments, and namespaces via kubectl & manifests
Infrastructure as Code (Terraform)	Declaratively provisioned K8s resources; managed Terraform state
CI/CD Automation (Azure DevOps)	Built end-to-end YAML pipelines from commit to cluster deployment
Container Registry Management	Tagged, versioned, and published images to Docker Hub
GitHub Version Control	Branch strategies, pull requests, and commit-triggered automation
Cloud-Native Deployment	Production-grade rolling updates with zero-downtime strategies
Observability Planning	Designed Prometheus/Grafana monitoring strategy for phase 7

This project delivers a fully functional, enterprise-grade DevOps platform that demonstrates the complete software delivery lifecycle — from source code to production-ready containerised microservices. Every component reflects the practices and tooling used by modern technology organisations at scale.

The platform was built without shortcuts: each phase built directly on the previous one, and the pipeline operates end-to-end with no manual steps. Azure DevOps provides a single pane of glass for build history, test results, deployment approvals, and environment management — giving the team full visibility and control over every release.

Project Summary

- 7 technology domains covered: development, containers, orchestration, IaC, CI/CD, registry, monitoring
- 3 independent microservices deployed and communicating on Kubernetes
- Fully automated pipeline — zero manual deployments after initial setup
- Infrastructure declared as code — every resource reproducible from a single terraform apply
- Production deployment patterns: rolling updates, health probes, replica sets
- Built for extensibility: Helm, ArgoCD, SonarQube enhancements planned for next phase

Enterprise Cloud-Native DevOps Platform

Prepared for Management Review | May 2026 | Confidential

Azure DevOps • Kubernetes • Docker • Terraform • GitHub